Blocking and Nonblocking Assignments

1 Introduction

Verilog has two procedural assignment operators: blocking (=) and nonblocking (<=). They can produce different simulation behavior, especially when several assignments execute on the same clock event. A useful starting convention is to use blocking assignments for combinational logic and nonblocking assignments for clocked, sequential logic.

The examples below also touch on counters, multiple clock edges, and generate loops. These topics appeared alongside the assignment examples in the original notes and are retained here, with the code presented as editable text rather than screenshots.

2 Combinational logic: an 8-to-1 multiplexer

A combinational block should assign its output for every possible input condition. The default branch below makes the behavior explicit when `sel` contains an unknown or high-impedance bit. In simulation, the output then becomes unknown (`x`), which can help expose an invalid or uninitialized select signal.

```
module mux_8to1 (in, sel, out);
  input [7:0] in;
  input [2:0] sel;
  output reg out;

  always @(*) begin
    case (sel)
      3'b000: out = in[0];
      3'b001: out = in[1];
      3'b010: out = in[2];
      3'b011: out = in[3];
      3'b100: out = in[4];
      3'b101: out = in[5];
      3'b110: out = in[6];
      3'b111: out = in[7];
      default: out = 1'bx;
    endcase
  end
endmodule
```

For synthesizable hardware, an `x` default can be treated as a don't-care by synthesis. Use it only when that is appropriate for the design; otherwise choose a defined safe output value.

3 Clocked counters and design parameters

3.1 Synchronous up/down counter

This example loads data, clears the count, increments, or decrements on the rising clock edge. The order of the `if`/`else if` branches establishes priority: load takes precedence over clear, and clear takes precedence over counting.

```
module counter (mode, clr, ld, d_in, clk, count);
  input mode, clr, ld, clk;
  input [7:0] d_in;
  output reg [7:0] count;

  always @(posedge clk) begin
    if (ld)
      count <= d_in;
    else if (clr)
      count <= 0;
    else if (mode)
      count <= count + 1;
    else
      count <= count - 1;
  end
endmodule
```

Because these are clock-triggered state updates, nonblocking assignments are used. This example has a synchronous clear: `clr` is checked only on the rising edge of `clk`.

3.2 Parameterized counter

A parameter lets the same module describe counters of different widths. Here `N` means the number of bits, so the vector is indexed `[N-1:0]`.

```
module counter #(parameter N = 8) (
  input clear,
  input clock,
  output reg [N-1:0] count
);
  always @(negedge clock) begin
    if (clear)
      count <= 0;
    else
      count <= count + 1'b1;
  end
endmodule
```

Parameter values are resolved during elaboration, before simulation or synthesis. The original example used a range ending at `N`, which creates `N+1` bits; the version above makes the parameter consistently represent the width.

4 Multiple clocks and clock edges

4.1 Separate clock domains

A module can contain separate sequential blocks triggered by different clocks or edges. Each block updates its own state at its specified event. When a signal crosses between unrelated clock domains, additional clock-domain-crossing design (for example, synchronizers or handshakes) is usually required; simply using two clocks does not make that transfer safe.

```
module multiple_clk (clk1, clk2, a, b, c, f1, f2);
  input clk1, clk2, a, b, c;
  output reg f1, f2;

  always @(posedge clk1)
    f1 <= a & b;

  always @(negedge clk2)
    f2 <= b ^ c;
endmodule
```

4.2 Both edges of one clock

The next example updates `f` on the rising edge and the intermediate register `t` on the falling edge. The same clock is used, but the two updates happen at different times in each cycle.

```
module multi_edge_clk (a, b, f, clk);
  input a, b, clk;
  output reg f;
  reg t;

  always @(posedge clk)
    f <= t & b;

  always @(negedge clk)
    t <= a | b;
endmodule
```

An operation launched on one edge and consumed on the opposite edge has only about half a cycle to settle. That timing assumption must be checked against the clock period, delays, and setup-time requirements.

The related arithmetic example computes a sum on the rising edge and a difference on the falling edge. The signal `c` must be an output register because the rising-edge block assigns it.

```
module multi_edge_arithmetic (a, b, d, clk, c, f);
  input [7:0] a, b, d;
  input clk;
  output reg [7:0] c, f;

  always @(posedge clk)
    c <= a + b;

  always @(negedge clk)
    f <= c - d;
endmodule
```

5 Ring counter: why assignment order matters

A ring counter circulates a bit pattern through a register. The sketch shows a four-stage version; the code examples use eight bits and circulate the most-significant bit back to bit zero.

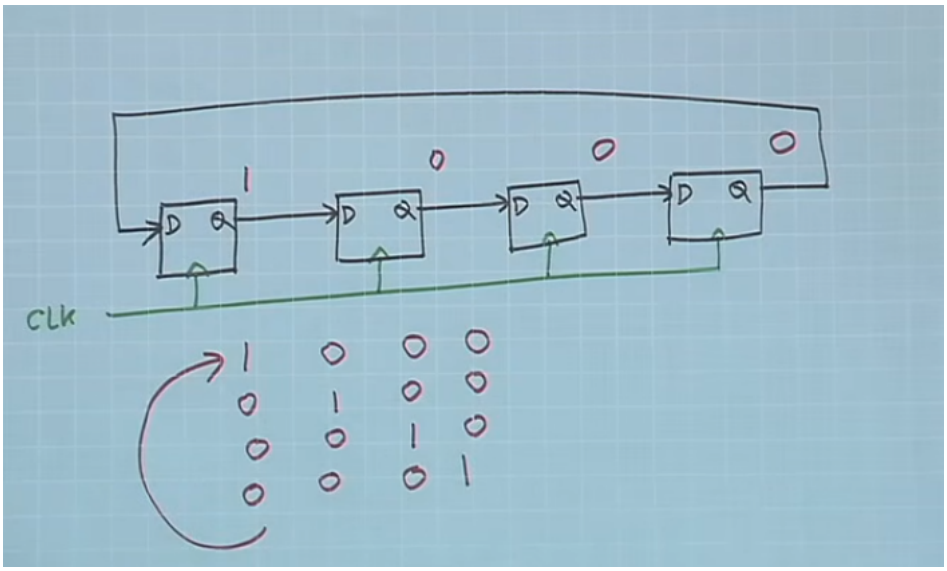


Figure 1: Ring-counter stages and the circulating one-hot pattern.

In the first version, blocking assignments update `count` immediately, one statement at a time. After the shift, bit 7 has already become zero, so the following statement copies zero back into bit 0. The intended wraparound is lost.

```
always @(posedge clk) begin
  if (init)
    count = 8'b10000000;
  else begin
    count = count << 1;
    count[0] = count[7];
  end
end
```

With nonblocking assignments, both right-hand sides are evaluated using the pre-edge value. A single concatenation makes the rotation especially clear and avoids overlapping updates to the same vector:

```
always @(posedge clk) begin
  if (init)
    count <= 8'b10000000;
  else
    count <= {count[6:0], count[7]};
end
```

The concatenation version describes the same rotation directly: old bits 6 through 0 move up one position, while old bit 7 wraps around into bit 0.

6 Blocking and nonblocking semantics

6.1 What the operators do

With a blocking assignment, the left-hand side is updated before the next statement in that procedural block executes. With a nonblocking assignment, the right-hand side is evaluated when the statement runs, but the left-hand-side update is scheduled for later in the current simulation time step (or a specified delay later). This lets several clocked statements all use the old state and then update together.

- Use = for combinational calculations in procedural blocks such as `always @(*)`.
- Use <= for registers updated by clocked blocks.
- A nonblocking assignment does not defer evaluation of its right-hand side; it defers only the update to the left-hand side.

6.2 Delayed assignments

In the first code block, each intra-assignment delay suspends execution until the delayed blocking update occurs. Consequently, the second right-hand side is read five time units after the block starts. With nonblocking assignments, both right-hand sides are read immediately, and both updates are scheduled five time units later.

```
begin
  a =  #5 b;
  c =  #5 a;
end
```

```
begin
  a <= #5 b;
  c <= #5 a;
end
```

Thus, `c` can receive different values in the two versions. Delays are useful for understanding simulation scheduling, but synthesizable RTL generally models timing through clocks and constraints rather than procedural delays.

6.3 Two clocked assignments in one block

These two statements, when written with blocking assignments, execute sequentially: `q2` sees the newly assigned value of `q1`. With nonblocking assignments, both right-hand sides use the values from just before the clock edge, which describes two cascaded flip-flops.

```
always @(posedge clock) begin
  q1 = a;
  q2 = q1;
end
```

```
always @(posedge clock) begin
  q1 <= a;
  q2 <= q1;
end
```

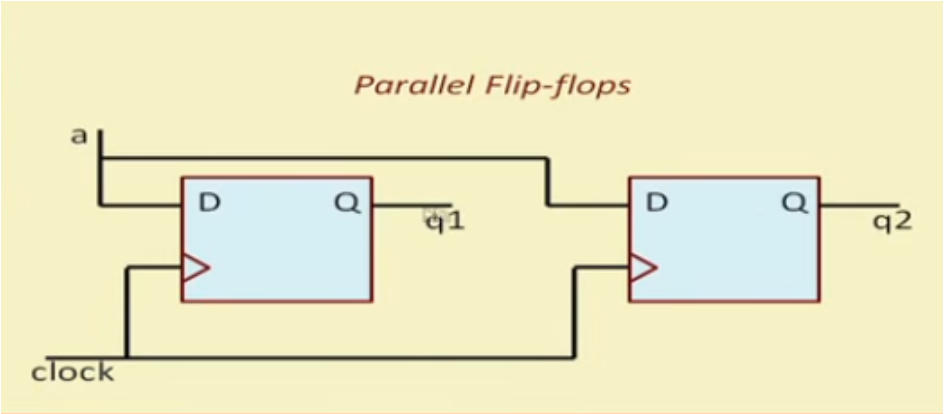


Figure 2: Two clocked stages: the second flip-flop samples the first stage's old value.

Reversing the order of blocking statements changes the result. Updating `q2` before `q1` lets `q2` read the old `q1`; nonblocking assignments retain that old-state behavior regardless of statement order.

```
always @(posedge clock) begin
  q2 = q1;
  q1 = a;
end
```

```
always @(posedge clock) begin
  q1 <= a;
  q2 <= q1;
end
```

6.4 Separate clocked blocks and race conditions

Separate `always` blocks triggered by the same edge can run in either order in simulation. If one block uses a blocking assignment to update a value that another block reads on that edge, the result can depend on which block runs first. This is a race, not a reliable way to describe a pipeline.

```
always @(posedge clock)
  q1 = a;

always @(posedge clock)
  q2 = q1;
```

Use nonblocking assignments for both clocked blocks so each register samples the old state:

```
always @(posedge clock)
  q1 <= a;

always @(posedge clock)
  q2 <= q1;
```

6.5 A four-bit shift register

The following asynchronous-clear shift register uses an active-low clear. Its assignments proceed from the last stage back toward the first, so the blocking version reads each stage before it is overwritten. Nevertheless, nonblocking assignments are the recommended style for sequential logic: they preserve old-state behavior without relying on statement order.

```
module shiftreg_4bit (clock, clear, A, E);
  input clock, clear, A;
  output reg E;
  reg B, C, D;

  always @(posedge clock or negedge clear) begin
    if (!clear) begin
      B = 0;
      C = 0;
      D = 0;
      E = 0;
    end else begin
      E = D;
      D = C;
      C = B;
      B = A;
    end
  end
endmodule
```

The recommended form changes the sequential assignments to nonblocking. Reset assignments are nonblocking as well:

```
always @(posedge clock or negedge clear) begin
  if (!clear) begin
    B <= 0;
    C <= 0;
    D <= 0;
    E <= 0;
  end else begin
    E <= D;
    D <= C;
    C <= B;
    B <= A;
  end
end
```

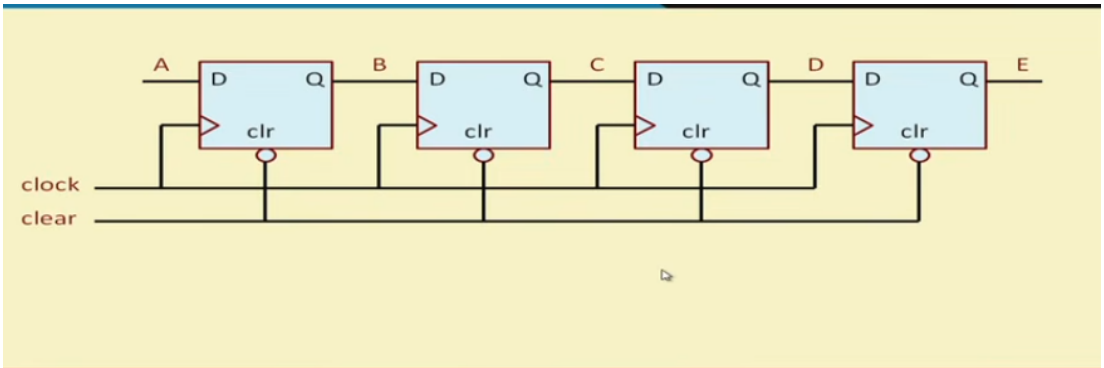


Figure 3: Four clocked stages, with a shared clock and active-low clear.

If the blocking assignments are reversed (`B = A`, then `C = B`, and so on through `E = D`), each statement reads the value written by the previous statement. All stages can then take the new input in the same event, rather than shifting it one stage per clock. Nonblocking assignments avoid that ordering effect. The essential storage behavior in this case reduces to a single stage:

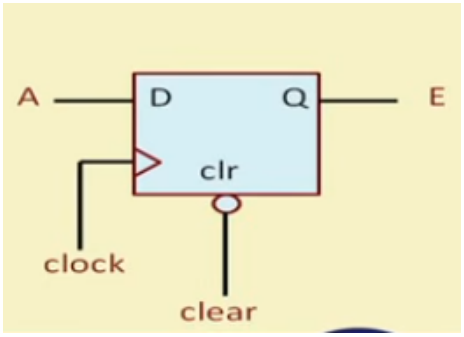


Figure 4: A single D flip-flop, rather than a multi-stage shift register.

7 Mixing assignment types

Mixing blocking and nonblocking assignments within one procedural block is legal, but can make simulation behavior difficult to follow. Keep combinational and sequential logic distinct when possible, and avoid mixing the operators in a clocked block.

The following small examples show that blocking and nonblocking versions can produce the same result when the procedural ordering and sampled values make the behavior unambiguous.

Example 1: The final value of `x` is 20, and `y` receives 20 in either version.

```
always @(*) begin
  x = 10;
  x = 20;
  y = x;
  #10;
end
```

```
always @(*) begin
  x = 10;
  x = 20;
  y <= x;
  #10;
end
```

Example 2: Here `y` receives 10 before `x` is changed to 20. In the nonblocking version the right-hand side is still sampled at the statement, even though the update to `y` is scheduled.

```
always @(*) begin
  x = 10;
  y = x;
  x = 20;
  #10;
end
```

```
always @(*) begin
  x = 10;
  y <= x;
  x = 20;
  #10;
end
```

These examples illustrate event scheduling; they are not a recommendation to use delays or nonblocking assignments in combinational logic. In ordinary RTL, use blocking assignments for combinational calculations and nonblocking assignments for clocked state.

8 Generate blocks

Generate constructs create repeated hardware during elaboration. They are useful for parameterized structures such as an N-bit bank of XOR gates or a ripple-carry adder. A `genvar` exists for elaboration

of the loop; it is not a run-time register.

8.1 Parameterized bitwise XOR

The named generate block `xorloop` gives each generated gate a predictable hierarchical path, such as `xorloop[0].XG` through `xorloop[N-1].XG`.

```
module xor_bitwise #(parameter N = 16) (
  input [N-1:0] a, b,
  output [N-1:0] f
);
  genvar p;
  generate
    for (p = 0; p < N; p = p + 1) begin : xorloop
      xor XG (f[p], a[p], b[p]);
    end
  endgenerate
endmodule
```

The following testbench applies two input pairs and prints the generated output with `$monitor`:

```
module generate_test;
  reg [15:0] x, y;
  wire [15:0] out;

  xor_bitwise G (.f(out), .a(x), .b(y));

  initial begin
    $monitor("x: %b, y: %b, out: %b", x, y, out);
    x = 16'hAAAA;
    y = 16'h00FF;
    #10 x = 16'h0F0F;
        y = 16'h3333;
    #20 $finish;
  end
endmodule
```

For example, with `a = 16'hAAAA` and `b = 16'h00FF`, the output is `16'hAA55`. With `a = 16'h0F0F` and `b = 16'h3333`, it is `16'h3C3C`. Each output bit is the XOR of the corresponding input bits.

8.2 Ripple-carry adder

The loop below creates one full-adder stage for each bit. The carry vector has $N + 1$ entries: `carry[0]` is the input carry, and `carry[N]` is the final carry-out. The name `fa_loop` labels each generated stage.

```
module RCA #(parameter N = 8) (
  output carry_out,
  output [N-1:0] sum,
  input [N-1:0] a, b,
  input carry_in
);
  wire [N:0] carry;
  assign carry[0] = carry_in;
  assign carry_out = carry[N];

  genvar i;
  generate
    for (i = 0; i < N; i = i + 1) begin : fa_loop
      wire t1, t2, t3;
      xor G1 (t1, a[i], b[i]);
      xor G2 (sum[i], t1, carry[i]);
      and G3 (t2, a[i], b[i]);
      and G4 (t3, t1, carry[i]);
      or  G5 (carry[i+1], t2, t3);
    end
  endgenerate
endmodule
```

Generated instance paths include `fa_loop[0].G1` and `fa_loop[1].G1`. The last-stage path follows the same pattern: `fa_loop[N-1].G1`. Local nets can likewise be referenced by paths such as `fa_loop[0].t1` and `fa_loop[1].t2` in a simulator or debugger.

9 Summary

- A blocking assignment (`=`) updates immediately within its procedural block; later statements see that update.
- A nonblocking assignment (`<=`) evaluates its right-hand side immediately and schedules the left-hand-side update, allowing clocked registers to use the same pre-edge state.
- Prefer blocking assignments in combinational procedural logic and nonblocking assignments in clocked sequential logic.
- Avoid blocking assignments across separate clocked processes when one process reads state another process writes on the same edge; this can create simulation races.
- Keep reset polarity, clock edge, parameter width, and assignment priority explicit in RTL.